Build Task — Tool Library — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

- **`tools.py`** — `AddArgs`, `WeatherArgs`, `LookupArgs` (Pydantic), the three `@tool(args_schema=...)` functions, and `ALL_TOOLS = [add, get_weather, flaky_lookup]`, so later documents import one list.
- **`flaky_lookup(query, should_fail=False)`** — `should_fail` makes the failure predictable for testing instead of random. Its description tells the model when to set it.
- **`tool_harness.py`** — a `ToolCallResult` dataclass (prompt, tool name, arguments, output, the model's final answer), `find_tool()`, `run_tool()` that turns a `ValidationError` into an error string, and `run_with_tools(prompt, tools)`, which runs every requested call and sends each result back before asking the model for its final answer.
- **`test_prompts.py`** — one prompt per Test Cases row, plus one for the real service.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
# tool_harness.py — the part that's new here
def run_tool(tools: list[BaseTool], call: dict) -> str:
    matching_tool = find_tool(tools, call["name"])
    if matching_tool is None:
```

```
        return "Error: tool not found"
    try:
        return str(matching_tool.invoke(call["args"]))
    except ValidationError:
        return "Error: the arguments failed the tool's checks."
```

Then `run_with_tools()` is the Failure exercise's round trip: append the model's response to `messages`, loop over `response.tool_calls`, append one tool message per call, and invoke again. Finish it, then compare against the Solution.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

`tools.py` is the same as the Basic version above.

Story — `tool_harness.py`: `run_tool()` turns bad arguments into an error string (Edge cases), and `run_with_tools()` sends every result back tagged with its `tool_call_id` and asks the model again (Failure) — so `final_answer` shows how the model really reacted. Every pick is logged with Doc01's `get_logger()`. **If not:** a failing tool's error would never reach the model, and one bad argument would crash the whole test run.

```
# tool_harness.py
from dataclasses import dataclass

# how: BaseTool is the type every @tool function has (the Failure
# exercise's count_tool_calls) — list[BaseTool] means "a list of tools"
from langchain_core.tools import BaseTool
from langchain_openai import ChatOpenAI
from pydantic import ValidationError

from logging_setup import get_logger

@dataclass
class ToolCallResult:
    prompt: str
    tool_name: str | None
    arguments: dict
    output: str
    final_answer: str

def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    # why: pulled out on its own so it's testable with plain tool objects
```

```
    # and a name string — no model call needed to prove it works.
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None

def run_tool(tools: list[BaseTool], call: dict) -> str:
    matching_tool = find_tool(tools, call["name"])
    if matching_tool is None:
        return "Error: tool not found"
    try:
        return str(matching_tool.invoke(call["args"]))
    except ValidationError:
        # why: bad arguments (an empty city) become a message the model
        # can read and react to — the Edge cases exercise's rule
        return "Error: the arguments failed the tool's checks."

def run_with_tools(prompt: str, tools: list[BaseTool]) -> ToolCallResult:
    logger = get_logger(__name__)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(tools)

    messages = [{"role": "user", "content": prompt}]
    response = model_with_tools.invoke(messages)

    if not response.tool_calls:
        # when: the model answered directly — a real, checkable outcome
        logger.info(f"no tool for: {prompt}")
        return ToolCallResult(prompt, None, {}, "", response.content)

    # how: the Failure exercise's round trip — the request, then one
    # tool message per call, then a second model call
    messages.append(response)
    outputs = []
    for call in response.tool_calls:
        logger.info(f"model picked {call['name']} with {call['args']}")
        output = run_tool(tools, call)
        outputs.append(output)
        messages.append(
            {"role": "tool", "content": output, "tool_call_id": call["id"]}
        )

    final = model_with_tools.invoke(messages)
    first_call = response.tool_calls[0]
    return ToolCallResult(
        prompt=prompt,
        tool_name=first_call["name"],
```

```
        arguments=first_call["args"],
        output=outputs[0],
        final_answer=final.content,
    )
```

Story — `test_prompts.py`: this Build Task's test file — one prompt per row of the Test Cases table, plus the real service, each printed on four short lines. **If not:** you'd ship the library never having watched it handle a missing argument or a failing tool.

```
# test_prompts.py
# Runs the 4 Test Cases from the README, plus the real-service tool.
# Run it from inside this folder: python test_prompts.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from tool_harness import run_with_tools
from tools import ALL_TOOLS

# how: (label, prompt) pairs — one per row of the Test Cases table
TEST_PROMPTS = [
    ("clear match", "what's 5 + 7?"),
    ("real service", "what's the weather in Lahore?"),
    ("no tool", "tell me a joke"),
    ("missing argument", "what's the weather like?"),
    ("tool fails", "look up the refund policy, and force the lookup to fail"),
]

def main() -> None:
    for label, prompt in TEST_PROMPTS:
        result = run_with_tools(prompt, ALL_TOOLS)
        print(f"{label}: {prompt}")
        print(f"  tool:   {result.tool_name} {result.arguments}")
        print(f"  output: {result.output}")
        print(f"  answer: {result.final_answer}")

if __name__ == "__main__":
    main()
```

Expected output (model wording varies):

```
clear match: what's 5 + 7?
  tool:   add {'a': 5, 'b': 7}
  output: 12
  answer: 5 + 7 is 12.
real service: what's the weather in Lahore?
  tool:   get_weather {'city': 'Lahore'}
  output: Currently 34.2°C in Lahore
```

```
  answer: It's currently 34.2°C in Lahore.
no tool: tell me a joke
  tool:   None {}
  output:
  answer: Why did the chicken cross the road? To get to the other side.
missing argument: what's the weather like?
  tool:   None {}
  output:
  answer: Which city would you like the weather for?
tool fails: look up the refund policy, and force the lookup to fail
  tool:   flaky_lookup {'query': 'refund policy', 'should_fail': True}
  output: Error: simulated failure
  answer: The lookup failed, so I couldn't get the refund policy.
```

For "missing argument" the model usually asks for the city. If it sends an empty city instead, you'll see `tool: get_weather {'city': ''}` and `output: Error: the arguments failed the tool's checks.` — both are a pass for that Test Case.